Artificial Intelligence Technology and Application

Deep Learning Lab Guide

Student Version

HUAWEI

HUAWEI TECHNOLOGIES CO., LTD

About This Document

Description

This guide introduces the following five exercises:

- Exercise 1: MindSpore basics, which describes the basic syntax and common modules of MindSpore.
- Exercise 2: Handwritten character recognition, in which the MindSpore framework is used to recognize handwritten characters.
- Exercise 3: MobileNetV2 image classification, which mainly introduces the classification of flower images using the lightweight network MobileNetV2.
- Exercise 4: ResNet-50 image classification exercise, which mainly introduces the classification of flower images using the ResNet-50 model.
- Exercise 5: TextCNN sentiment analysis, which mainly introduces sentiment analysis of statements using the TextCNN model.

Background Knowledge Required

This course is for Huawei's basic certification. To better understand this course, familiarize yourself with the following:

- Basic knowledge of Python, MindSpore concepts, and Python programming.

Contents

About This Document 1
1 MNIST Handwritten Character Recognition 3
1.1 Introduction 3
1.1.1 About This Exercise 3
1.2 Preparations 3
1.3 Detailed Design and Implementation 3
1.3.1 Data Preparation 3
1.3.2 Procedure 4

1 MNIST Handwritten Character Recognition

1.1 Introduction

1.1.1 About This Exercise

This exercise implements the MNIST handwritten character recognition, which is a typical case in the deep learning field. The whole process is as follows:

- Process the required dataset. (The MNIST dataset is used in this example.)
- Define a network. (A simple fully-connected network is built in this example.)
- Define a loss function and an optimizer.
- Load the dataset and perform training. After the training is complete, use the test set for validation.

1.2 Preparations

Before you start, check whether MindSpore has been correctly installed. You are advised to install MindSpore on your computer by referring to the MindSpore official website https://www.mindspore.cn/install/en.

In addition, you should have basic mathematical knowledge, including knowledge of Python coding basics, probability, and matrices.

Recommended environment:

Version: MindSpore 1.7

Programming language: Python 3.7

1.3 Detailed Design and Implementation

1.3.1 Data Preparation

The MNIST dataset used in this example consists of 10 classes of 28 x 28 pixels grayscale images. It has a training set of 60,000 examples, and a test set of 10,000 examples.

Download the MNIST dataset at http://yann.lecun.com/exdb/mnist/ (OBS: https://certification-data.obs.cn-north-4.myhuaweicloud.com/CHS/HCIA-AI/V3.5/chapter4/MNIST.zip). Four dataset download links are provided. The first two

links are for downloading test data files, and the last two links are for downloading training data files.

Download and decompress the files, and store them in the workspace directories **./MNIST /train** and **./MNIST /test**.

The directory structure is as follows:

```
└─MNIST
    ├─ test
    │      t10k-images.idx3-ubyte
    │      t10k-labels.idx1-ubyte
    │
    └─ train
           train-images.idx3-ubyte
           train-labels.idx1-ubyte
```

1.3.2 Procedure

Step 1 Import the Python library and module and configure running information.

Import the required Python library.

Currently, the **os** library is required. Other required libraries will not be described here. For details about the MindSpore modules, see the MindSpore API page. You can use context.set_context to configure the information required for running, such as the running mode, backend information, and hardware information.

Import the context module and configure the required information.

Code:

```
# Import related dependent libraries.
import   os
from matplotlib import pyplot as plt
import numpy as np

import mindspore as ms
import mindspore.context as context
import mindspore.dataset as ds
import mindspore.dataset.transforms.c_transforms as C
import mindspore.dataset.vision.c_transforms as CV
from mindspore.nn.metrics import Accuracy

from mindspore import nn
from mindspore.train import Model
from mindspore.train.callback import ModelCheckpoint, CheckpointConfig, LossMonitor, TimeMonitor

context.set_context(mode=context.GRAPH_MODE, device_target='CPU')
```

The graph mode is used in this exercise. You can configure hardware information as required. For example, if the code runs on the Ascend AI processor, set **device_target** to

Ascend. This rule also applies to the code running on the CPU and GPU. For details about parameters, see the context.set_context API description at https://www.mindspore.cn/docs/en/r1.7/api_python/mindspore.context.html.

Step 2 Read data.

Use the data reading function of MindSpore to read the MNIST dataset and view the data volume and sample information of the training set and test set.

Code:

```
DATA_DIR_TRAIN = "MNIST/train" # Training set information
DATA_DIR_TEST = "MNIST/test" # Test set information
# Read data.


# Display the dataset features.
print('Data volume of the training dataset:',ds_train.get_dataset_size())
print('Data volume of the test dataset:',ds_test.get_dataset_size())
image=ds_train.create_dict_iterator().__next__()
print('Image length/width/channels:',image['image'].shape)
print('Image label style:',image['label'])      # Total 10 label classes which are represented by
numbers from 0 to 9.
```

Step 3 Process data.

Datasets are crucial for training. A good dataset can effectively improve training accuracy and efficiency. Generally, before loading a dataset, you need to perform some operations on the dataset.

Define a dataset and data operations.

We define the create_dataset function to create a dataset. In this function, we define the data augmentation and processing operations to be performed:

- Read the dataset.
- Define parameters required for data augmentation and processing.
- Generate corresponding data augmentation operations according to the parameters.
- Use the map function to apply data operations to the dataset.
- Process the generated dataset.

Code:

```
def create_dataset(training=True, batch_size=128, resize=(28, 28),
                           rescale=1/255, shift=0, buffer_size=64):
    ds = ms.dataset.MnistDataset(DATA_DIR_TRAIN if training else DATA_DIR_TEST)
    # Define the resizing, normalization, and channel conversion of the map operation.



    # Perform the map operation on the dataset.
```

```
# Set the shuffle parameter and batch size.
```

In the preceding information, **batch_size** indicates the number of data records in each batch. Assume that each batch contains 32 data records. Modify the image size, normalization, and image channel, and then modify the data type of the label. Perform the shuffle operation, set **batch_size**, and set **drop_remainder** to **True**. In this case, data that cannot form a batch in the dataset will be discarded.

MindSpore supports multiple data processing and augmentation operations, which are usually used together. For details, see **Data Processing and Augmentation** on the MindSpore official website.

Step 4 Sample visualization

Read the first 10 samples and visualize the samples to determine whether the samples are real datasets.

Code:

```
# Display the first 10 images and the labels, and check whether the images are correctly labeled.
```

Output:

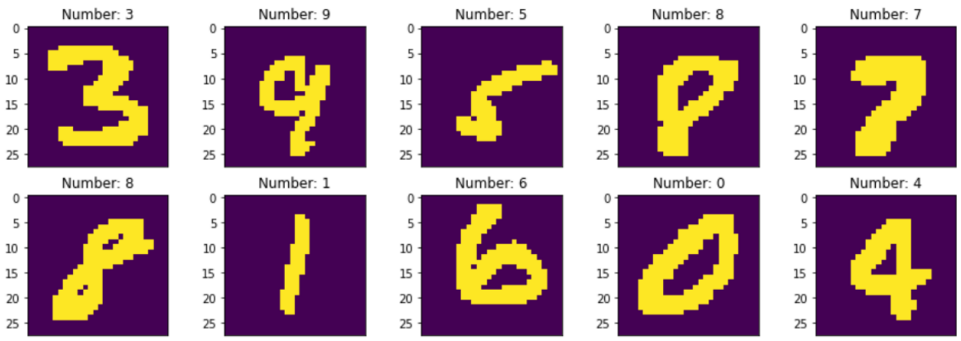

Figure 1-1 Sample visualization

Step 5 Define a network.

We define a simple fully-connected network to implement image recognition. The network has only three layers:

The first layer is a fully-connected layer of the shape 784 x 512.

The second layer is a fully-connected layer of the shape 512 x 128.

The last layer is an output layer of the shape 128 x 10.

To use MindSpore for neural network definition, inherit mindspore.nn.Cell. Cell is the base class of all neural networks (such as Conv2d).

Define each layer of a neural network in the __init__ method in advance, and then define the construct method to complete the feedforward construction of the neural network. The network layers are defined as follows:

Code:

```
# Create a model. The model consists of three fully connected layers. The final output layer uses
#softmax for classification (10 classes consisting of numbers 0 to 9.)
```

Step 6 Define a loss function and an optimizer.

A loss function is also called an objective function and is used to measure the difference between a predicted value and an actual value. Deep learning reduces the loss value by continuous iteration. Defining a good loss function can effectively improve model performance.

An optimizer is used to minimize the loss function, improving the model during training.

After the loss function is defined, the weight-related gradient of the loss function can be obtained. The gradient is used to indicate the weight optimization direction for the optimizer, improving model performance. Loss functions supported by MindSpore include SoftmaxCrossEntropyWithLogits, L1Loss, and MSELoss. SoftmaxCrossEntropyWithLogits is used in this example.

MindSpore provides the callback mechanism to execute custom logic during training. The following uses ModelCheckpoint provided by the framework as an example. ModelCheckpoint can save the network model and parameters for subsequent fine-tuning.

Code:

```
# Create a network, loss function, validation metric, and optimizer, and set related hyperparameters.
```

Step 7 Start training.

The training process refers to a process in which a training dataset is transferred to a network for training and optimizing network parameters. In the MindSpore framework, the Model.train method is used to complete this process.

Code:

```
# Build a model.



# Generate a dataset.



# Train the model.
```

Although loss values may fluctuate, they gradually decrease and the accuracy gradually increases in general. Loss values displayed each time may be different because of their randomicity. The following is an example of loss values output during training:

```
============== Starting Training ==============
epoch: 1 step: 1875, loss is 0.06333521
epoch time: 18669.680 ms, per step time: 9.957 ms
epoch: 2 step: 1875, loss is 0.07061358
epoch time: 21463.662 ms, per step time: 11.447 ms
epoch: 3 step: 1875, loss is 0.043515638
epoch time: 25836.919 ms, per step time: 13.780 ms
epoch: 4 step: 1875, loss is 0.03468642
epoch time: 25553.150 ms, per step time: 13.628 ms
epoch: 5 step: 1875, loss is 0.03934026
epoch time: 27364.246 ms, per step time: 14.594 ms
epoch: 6 step: 1875, loss is 0.0023852987
epoch time: 31432.281 ms, per step time: 16.764 ms
epoch: 7 step: 1875, loss is 0.010915326
epoch time: 33697.183 ms, per step time: 17.972 ms
epoch: 8 step: 1875, loss is 0.011417691
epoch time: 29594.438 ms, per step time: 15.784 ms
epoch: 9 step: 1875, loss is 0.00044568744
epoch time: 28676.948 ms, per step time: 15.294 ms
epoch: 10 step: 1875, loss is 0.071476705
epoch time: 34999.863 ms, per step time: 18.667 ms
```

Step 8 Validate the model.

In this step, the original test set is used to validate the model.

Code:

```
# Use the test set to validate the model and print the overall accuracy.
metrics=model.eval(ds_eval)
print(metrics)
```

Output:

```
{'Accuracy': 0.9740584935897436}
```